



Parte 1. Sintaxis de LOCK TABLES y UNLOCK TABLES

```
LOCK TABLES tbl_name [AS alias] {READ [LOCAL] | [LOW_PRIORITY] WRITE} [,
    tbl_name [AS alias] {READ [LOCAL] | [LOW_PRIORITY] WRITE}] ...
```

UNLOCK TABLES

LOCK TABLES bloquea tablas para el flujo actual. Si alguna de las tablas la bloquea otro flujo, bloquea hasta que pueden adquirirse todos los bloqueos. UNLOCK TABLES libera cualquier bloqueo realizado por el flujo actual. Todas las tablas bloqueadas por el flujo actual se liberan implícitamente cuando el flujo realiza otro LOCK TABLES, o cuando la conexión con el servidor se cierra.

Un bloqueo de tabla protege sólo contra lecturas inapropiadas o escrituras de otros clientes. El cliente que tenga el bloqueo, incluso un bloqueo de lectura, puede realizar operaciones a nivel de tabla tales como DROP TABLE.

Tenga en cuenta lo siguiente a pesar del uso de LOCK TABLES con tablas transaccionales:

- LOCK TABLES no es una operación transaccional y hace un commit implícito de cualquier transacción activa antes de tratar de bloquear las tablas. También, comenzar una transacción (por ejemplo, con START TRANSACTION) realiza un UNLOCK TABLES implícito. (Consulte Sección 13.4.3, “Sentencias que causan una ejecución (commit) implícita”.)
- La forma correcta de usar LOCK TABLES con tablas transaccionales, como InnoDB, es poner AUTOCOMMIT = 0 y no llamar a UNLOCK TABLES hasta que hace un commit de la transacción explícitamente. Cuando llama a LOCK TABLES, InnoDB internamente realiza su propio bloqueo de tabla, y MariaDB realiza su propio bloqueo de tabla. InnoDB libera su bloqueo de tabla en el siguiente commit, pero para que MariaDB libere su bloqueo de tabla, debe llamar a UNLOCK TABLES. No debe tener AUTOCOMMIT = 1, porque entonces InnoDB libera su bloqueo de tabla inmediatamente tras la llamada de LOCK TABLES, y los deadlocks pueden ocurrir fácilmente. (Tenga en cuenta que en MariaDB, no adquirimos el bloqueo de tabla InnoDB en absoluto si AUTOCOMMIT=1, para ayudar a aplicaciones antiguas a evitar deadlocks.)
- ROLLBACK no libera bloqueos de tablas no transaccionales de MariaDB.

Para usar LOCK TABLES en MariaDB, debe tener el permiso LOCK TABLES y el permiso SELECT para las tablas involucradas.

La razón principal para usar LOCK TABLES es para emular transacciones o para obtener más velocidad al actualizar tablas. Esto se explica con más detalle posteriormente.



Aula Virtual de Administración de Bases de Datos
Guía de Bloqueo de Tablas en MariaDB – Unidad 2



Si un flujo obtiene un bloqueo READ en una tabla, ese flujo (y todos los otros) sólo pueden leer de la tabla. Si un flujo obtiene un bloqueo WRITE en una tabla, sólo el flujo con el bloqueo puede escribir a la tabla. El resto de flujos se bloquean hasta que se libera el bloqueo.

La diferencia entre READ LOCAL y READ es que READ LOCAL permite comandos INSERT no conflictivos (inserciones concurrentes) se ejecuten mientras se mantiene el bloqueo. Sin embargo, esto no puede usarse si va a manipular los ficheros de base de datos fuera de MariaDB mientras mantiene el bloqueo. Para tablas InnoDB, READ LOCAL esencialmente no hace nada: No bloquea la tabla. Para tablas InnoDB, el uso de READ LOCAL está obsoleto ya que una SELECT consistente hace lo mismo, y no se necesitan bloqueos.

Cuando usa LOCK TABLES, debe bloquear todas las tablas que va a usar en sus consultas. Mientras los bloqueos obtenidos con un comando LOCK TABLES están en efecto, no puede acceder a ninguna tabla que no estuviera bloqueada por el comando. Además, no puede usar una tabla bloqueada varias veces en una consulta --- use alias para ello. Tenga en cuenta que, en este caso, debe tener un bloqueo separado para cada alias.

```
mariadb> LOCK TABLE t WRITE, t AS t1 WRITE;  
  
mariadb> INSERT INTO t SELECT * FROM t;  
  
ERROR 1100: Table 't' was not locked with LOCK TABLES  
  
mariadb> INSERT INTO t SELECT * FROM t AS t1;
```

Si sus consultas se refieren a una tabla que use un alias, debe bloquear la tabla que usa el mismo alias. No funciona bloquear la tabla sin especificar el alias:

```
mariadb> LOCK TABLE t READ;  
  
mariadb> SELECT * FROM t AS myalias;  
  
ERROR 1100: Table 'myalias' was not locked with LOCK TABLES
```

Si bloquea una tabla usando un alias, debe referirse a ella en sus consultas usando este alias:

```
mariadb> LOCK TABLE t AS myalias READ;  
  
mariadb> SELECT * FROM t;  
  
ERROR 1100: Table 't' was not locked with LOCK TABLES
```



```
mariadb> SELECT * FROM t AS myalias;
```

WRITE bloquea normalmente teniendo una prioridad superior que READ al bloquear para asegurar que las actualizaciones se procesan en cuanto se puede. Esto significa que, si un flujo obtiene un bloqueo READ y luego otro flujo pide un bloqueo WRITE, las peticiones de bloqueo READ posteriores esperan hasta que el flujo WRITE quita el bloqueo. Puede usar bloqueos LOW_PRIORITY WRITE para permitir a otros flujos que obtengan bloqueos READ mientras el flujo está en espera para el bloqueo WRITE. Debe usar bloqueos LOW_PRIORITY WRITE sólo si está seguro que habrá un momento sin flujos con bloqueos READ.

LOCK TABLES funciona como sigue:

Ordena todas las tablas a ser bloqueadas en un orden definido internamente. Desde el punto de vista del usuario, este orden es indefinido.

Si una tabla se bloquea con bloqueo de escritura y lectura, pone el bloqueo de lectura antes del de escritura.

Bloquea una tabla cada vez hasta que el flujo obtiene todos los bloqueos.

Esta política asegura un bloqueo de tablas libre de deadlocks. Sin embargo, hay otros puntos que debe tener en cuenta respecto a esta política:

Si está usando un bloqueo LOW_PRIORITY WRITE para una tabla, sólo significa que MariaDB espera para este bloqueo particular hasta que no hay flujos que quieren un bloqueo READ. Cuando el flujo ha obtenido el bloqueo WRITE y está esperando para obtener un bloqueo para la siguiente tabla en la lista, todos los otros flujos esperan hasta que el bloqueo WRITE se libera. Si esto es un problema con su aplicación, debe considerar convertir algunas de sus tablas a transaccionales.

Puede usar KILL para terminar un flujo que está esperando para un bloqueo de tabla.

Tenga en cuenta que no debe bloquear ninguna tabla que esté usando con INSERT DELAYED ya que en tal caso el INSERT lo realiza un flujo separado.

Normalmente, no tiene que bloquear tablas, ya que todos los comandos UPDATE son atómicos, ningún otro flujo puede interferir con ningún otro que está ejecutando comandos SQL. Hay algunos casos en que no debe bloquear tablas de ningún modo:

Si va a ejecutar varias operaciones en un conjunto de tablas MyISAM, es mucho más rápido bloquear las tablas que va a usar. Bloquear tablas MyISAM acelera la inserción, las actualizaciones, y los borrados. Por contra, ningún flujo puede actualizar una tabla con un bloqueo READ (incluyendo el



que tiene el bloqueo) y ningún flujo puede acceder a una tabla con un bloqueo WRITE distinto al que tiene el bloqueo.

La razón que algunas operaciones MyISAM sean más rápidas bajo LOCK TABLES es que MariaDB no vuelca la caché de claves para la tabla bloqueada hasta que se llama a UNLOCK TABLES. Normalmente, la caché de claves se vuelca tras cada comando SQL.

Si usa un motor de almacenamiento en MariaDB que no soporta transacciones, debe usar LOCK TABLES si quiere asegurarse que ningún otro flujo se ejecute entre un SELECT y un UPDATE. El ejemplo mostrado necesita LOCK TABLES para ejecutarse sin problemas:

```
mariadb> LOCK TABLES trans READ, customer WRITE;  
  
mariadb> SELECT SUM(value) FROM trans WHERE customer_id=some_id;  
  
mariadb> UPDATE custome SET total_value=sum_from_previous_statement  
  
WHERE customer_id=some_id;  
  
mariadb> UNLOCK TABLES;
```

Sin LOCK TABLES, es posible que otro flujo pueda insertar un nuevo registro en la tabla trans entre la ejecución del comando SELECT y UPDATE.

Puede evitar usar LOCK TABLES en varios casos usando actualizaciones relativas (UPDATE customer SET value=value+new_value) o la función LAST_INSERT_ID().

Puede evitar bloquear tablas en algunos casos usando las funciones de bloqueo de nivel de usuario GET_LOCK() y RELEASE_LOCK(). Estos bloqueos se guardan en una tabla hash en el servidor e implementa pthread_mutex_lock() y pthread_mutex_unlock() para alta velocidad.

Consulte “Métodos de bloqueo”, para más información acerca de la política de bloqueo.

Puede bloquear todas las tablas en todas las bases de datos con bloqueos de lectura con el comando FLUSH TABLES WITH READ LOCK. Esta es una forma muy conveniente para obtener copias de seguridad.

Nota: Si usa ALTER TABLE en una tabla bloqueada, puede desbloquearse.



Parte 2. Bloqueo de tablas en MariaDB

<https://blogprog.gonzalolopez.es/articulos/bloqueo-de-tablas.html>

MariaDB permite **bloquear tablas** para que diversas queries puedan ejecutarse al mismo tiempo sin distorsionar sus resultados. Este bloqueo puede ser de **escritura** o de **lectura**. Después de realizar el bloqueo hay que realizar un desbloqueo, como es lógico.

Para usar **LOCK TABLES** en MariaDB se necesita el privilegio global **LOCK TABLES** y el privilegio **SELECT** en las tablas involucradas. Ahora voy a poner la sintaxis para cada tipo de bloqueo:

- **Bloqueo por escritura: LOCK TABLES WRITE**

```
LOCK TABLES tbl_contador WRITE;  
  
UPDATE tbl_contador SET valor = valor + 1; //incrementar el contador  
  
SELECT valor FROM contador; //retornar el contador asignado  
  
UNLOCK TABLES;
```

- **Bloqueo por lectura: LOCK TABLES READ**

```
LOCK TABLES trans READ;  
  
SELECT SUM(value) FROM trans WHERE customer_id=some_id;  
  
UPDATE customer SET total_value=sum_from_previous_statement  
    WHERE customer_id=some_id;  
  
UNLOCK TABLES;
```

El bloqueo **WRITE** tiene mayor prioridad que el bloqueo **READ**, para asegurar que las actualizaciones se procesan lo antes como sea posible. Esto significa que si un hilo obtiene un bloqueo **READ** y después otro hilo realiza un bloqueo **WRITE**, las siguientes peticiones de bloqueos **READ** no se ejecutarán hasta que el hilo que solicitó el bloqueo **WRITE** lo libere. Se puede usar el bloqueo **LOW_PRIORITY WRITE** para que otros hilos puedan realizar bloqueos **READ** mientras el hilo espera al bloqueo **WRITE**.

Sólo se debe usar el bloqueo **LOW_PRIORITY WRITE** si se está seguro de que existirá un tiempo en el que no habrá hilos que pidan un bloqueo **READ**.